

【数据结构】栈(Stack)的链表实现

Tags:

Time Limit: 1 s Memory Limit: 128 MB
Submission: 302 AC: 81 Score: 96.13

Submit

Codes

AI Code Tips

Description

链表的头插法，如字面意思，每次新插入的节点位置都在头节点的后面。具体操作过程如下：

插入操作：

1. 新建节点NewNode
2. NewNode->next指向Head->next
3. Head->next指向NewNode

删除操作：

1. 节点指针TempPtr指向Head->next
2. Head->next指向Head->next->next
3. 释放TempPtr

Tips:链表有两种写法，一种是有头节点的链表，另外一种是没有头节点的链表，两者在插入操作中没有区别，但是在删除操作中，没有头节点的链表需要特判，而有头节点的链表不需要。

现在希望你对于给定的Stack结构体完成以下几类操作：

1. Status Push(LinkStack *S,SElemType *e) //将e压入栈中
2. Status Pop(LinkStack *S,SElemType *e) //将栈顶元素从栈中取出
3. Status StackTraverse(LinkStack *S) //从上到下输出栈内元素

详细要求如下：

1. 对于S，将e压入栈中，成为栈顶元素
2. 对于S，将该栈中的栈顶元素取出

3. 对于S，自上向下输出栈内元素，每个元素之间用空格隔开，若栈空则输出空行

Tips:主体代码已给出，你只需要提交函数

```
1#include <stdio.h>
2#include <string.h>
3#include <math.h>
4#include <stdlib.h>
5
6#define OK 1
7#define ERROR 0
8#define TRUE 1
9#define FALSE 0
10
11
12typedef int Status;
13typedef int SElemType;
14typedef struct SNode SNode;
15typedef struct SNode* SNptr;
16
17struct SNode
18{
19    SElemType data;
20    SNptr next;
21};
22
23typedef struct
24{
25    SNptr top;
26    int count;
27}LinkStack;
28
29Status LinkStackInit(LinkStack *s)
30{
31    s->top = NULL;
32    s->top = (SNptr)malloc(sizeof (SNode));
33    if (!s->top)return FALSE;
34    s->top->next = NULL;
35    s->count = 0;
36    return OK;
37}
38
39Status LinkStackClear(LinkStack *s)
40{
41    SNptr p = s->top->next,tmp;
42    while (p)
43    {
44        tmp = p;
45        p = p->next;
46        free(tmp);
47    }
48    s->count = 0;
49    return OK;
50}
51
52Status StackIsEmpty(LinkStack *s)
53{
```

```

54 if (s->count)return FALSE;
55 else return TRUE;
56}
57
58int StackSize(LinkStack *s)
59{
60 return s->count;
61}
62
63Status StackTop(LinkStack *s, SElemType *e)
64{
65 if (!s->top)return FALSE;
66 *e = s->top->data;
67 return TRUE;
68}
69Status StackVisit(SNptr s)
70{
71 if (!s)return FALSE;
72 printf("%d", s->data);
73 return OK;
74}
75
76Status StackPush(LinkStack *s, SElemType e);
77Status StackPop(LinkStack *s);
78Status StackTraverse(LinkStack *S);
79
80int main()
81{
82 int n;
83 char ctl[10];
84 LinkStack ss;
85 LinkStack *ssp = &ss;
86 ssp->top = NULL;
87 LinkStackInit(ssp);
88 scanf("%d", &n);
89 while (n--)
90 {
91 scanf("%s", ctl);
92 if (strcmp(ctl, "Push") == 0)
93 {
94 SElemType e;
95 scanf("%d", &e);
96 StackPush(ssp,e);
97 }
98 else if (strcmp(ctl, "Pop") == 0)
99 {
100 SElemType e;
101 if (StackIsEmpty(ssp))printf("ERROR: Stack is empty!\n");
102 else
103 {
104 StackTop(ssp, &e);
105 StackPop(ssp);
106 printf("%d\n", e);
107 }
108 }
109 else if (strcmp(ctl, "ShowAll") == 0)
110 {
111 StackTraverse(ssp);
112 }
113 }
114}

```

Input

Samples

input	Copy
<pre>8 Push 3 Push 2 Pop Pop Pop Push 6 Push 5 ShowAll</pre>	
output	Copy
<pre>2 3 ERROR: Stack is empty! 5 6</pre>	

SubmitCodes